

CS1003 Programming in C - Complete Study Notes

Test Preparation Guide for Modules 1 & 2

▮ MODULE 1: C PROGRAM FUNDAMENTALS

1. Structure of C Program

Complete C Program Structure (Sequential Order)

```
/* 1. Documentation Section */
// Program name, author, date, description

/* 2. Link Section */
#include <stdio.h>
#include <math.h>

/* 3. Definition Section */
#define PI 3.14159
#define MAX 100

/* 4. Global Declaration Section */
int global_var = 10;
float global_array[50];

/* 5. main() Function Section */
int main()
{
    /* Declaration Part */
    int local_var;
    float result;

    /* Executable Part */
    printf("Hello World\n");
    return 0;
}

/* 6. Subprogram Section */
// User-defined functions
void myFunction() {
    // Function code
}
```

Key Points to Remember:

- ▮ main() function is **mandatory**
- ▮ Documentation section is optional but recommended

- `#include` Link section contains header files
- `#define` Definition section contains macros
- Global declarations are accessible throughout the program
- Subprograms are user-defined functions

2. Memory Layout of C Program

Complete Memory Segments Table

Segment	Content Stored	Characteristics	Growth Direction	Example
Text/Code	- Machine code - Read-only data - String literals	- Read-only - Sharable - Fixed size	Static	<code>printf("Hello")</code> - string stored here
Initialized Data	- Initialized globals - Initialized statics - Constants	- Read-write area - Read-only area (const)	Static	<pre>int x = 5; (global) static int y = 10;</pre>
BSS (Block Started by Symbol)	- Uninitialized globals - Uninitialized statics	- Initialized to 0 - Takes no space in executable	Static	<pre>int arr[100]; (global) static int x;</pre>
Heap	- Dynamic memory <code>malloc/calloc</code> allocations	- Runtime allocation - Manual management	Grows Upward	<code>int *p = malloc(sizeof(int));</code>
Stack	- Local variables - Function parameters - Return addresses	- LIFO structure - Automatic management	Grows Downward	<pre>int main() { int x; }</pre>

Memory Segment Examples

```
#include <stdio.h>
#include <stdlib.h>

/* INITIALIZED DATA SEGMENT */
int global_init = 100;           // Initialized global
const char msg[] = "Hello";     // Read-only area
```

```

static int static_init = 200;    // Initialized static

/* BSS SEGMENT */
int global_uninit;               // Uninitialized global (set to 0)
static int static_uninit;       // Uninitialized static (set to 0)

int main() {
    /* STACK SEGMENT */
    int local_var = 50;          // Local variable
    char local_array[100];       // Local array

    /* HEAP SEGMENT */
    int *heap_var = (int*)malloc(sizeof(int));
    *heap_var = 300;

    /* TEXT SEGMENT */
    // The code itself is stored here

    free(heap_var);
    return 0;
}

```

Important Memory Facts:

- Stack and Heap grow towards each other
- When they meet → **Stack Overflow** or **Out of Memory**
- Text segment is often shared between multiple program instances
- BSS doesn't increase executable file size (just recorded as size)

3. C Program Compilation Process

Complete Compilation Pipeline

Phase	Input	Output	Tool	Function	Example Command
1. Preprocessing	.c file	Expanded .i file	Preprocessor	- Includes headers - Expands macros - Removes comments	gcc -E file.c
2. Compilation	.i file	Assembly .s file	Compiler	- Syntax checking - Generates assembly	gcc -S file.c
3. Assembly	.s file	Object .o/.obj file	Assembler	- Converts to machine code - Creates object code	gcc -c file.c

4. Linking	.o files + libraries	Executable .exe	Linker	• Links object files • Resolves references • Links libraries	gcc file.o -o program
5. Loading	.exe file	Running process	Loader	• Loads into RAM • Sets program counter	./program

Detailed Phase Operations:

Preprocessing Operations:

- `#include` → File inclusion
- `#define` → Macro substitution
- `#ifdef` → Conditional compilation
- `//` Comments → Removed
- `\n` Line continuations → Merged

Compilation Checks:

- `;` Syntax errors
- `;` Type checking
- `;` Function declarations
- `;` Variable scope

Linking Types:

- **Static Linking:** Libraries copied into executable
- **Dynamic Linking:** Libraries loaded at runtime

4. Library Functions

Key Points:

- `;` **Built-in functions** grouped in libraries
- `;` Located in header files (`.h`)
- `;` Pre-compiled and optimized
- `;` No need to write implementation

Common Library Functions:

Header File	Common Functions	Purpose
stdio.h	printf(), scanf(), fopen(), fclose()	Input/Output operations
stdlib.h	malloc(), free(), exit(), rand()	Memory & utility functions
string.h	strlen(), strcpy(), strcmp()	String operations
math.h	sqrt(), pow(), sin(), cos()	Mathematical operations
ctype.h	isalpha(), isdigit(), toupper()	Character operations

MODULE 2: DATA CONCEPTS & OPERATORS

1. Constants in C

Types of Constants

Constant Type	Examples	Declaration Methods	Memory Location
Integer Literal	10, -5, 0xFF (hex), 077 (octal)	Direct use	Text segment
Float Literal	3.14, -0.678, 2.5e3	Direct use	Text segment
Character Literal	'a', 'Z', '\n', '\t'	Single quotes	Text segment
String Literal	"Hello", "RV University"	Double quotes	Text segment
Symbolic Constant	#define PI 3.14159	Preprocessor directive	Replaced at compile time
const Variable	const int MAX = 100;	const keyword	Data segment

Important Constant Rules:

- Constants **cannot** be modified during execution
- String literals are stored in **read-only** memory
- `#define` creates compile-time constants (no memory)
- `const` creates runtime constants (uses memory)

2. Variables in C

Variable Types Comparison

Aspect	Local Variable	Global Variable	Static Variable
Declaration Location	Inside function/block	Outside all functions	With static keyword
Scope	Within declaring block	Entire program	Depends on location
Lifetime	Function execution	Program execution	Program execution
Default Value	Garbage value	0 (zero)	0 (zero)
Memory Segment	Stack	Data/BSS	Data/BSS
Access Speed	Fastest	Slower	Slower

Variable Naming Rules (STRICT)

Rule	Valid Examples	Invalid Examples	Reason
Start with letter/underscore	age, _value, Name	1st, 99names	Cannot start with digit
Contain letters, digits, underscore	student1, total_sum	my-var, cost\$	No special characters
Case sensitive	age ≠ Age ≠ AGE	-	Different variables
Not a keyword	integer, floating	int, for, while	Reserved words
Length	studentMarks (clear)	a (unclear), very long names	Readability

3. Data Types - Complete Reference

Primary Data Types

Data Type	Size (bytes)	Range	Format Specifier	Usage
char	1	-128 to 127	%c	Single character
unsigned char	1	0 to 255	%c	Positive characters
short int	2	-32,768 to 32,767	%hd	Small integers
unsigned short	2	0 to 65,535	%hu	Small positive integers
int	2 or 4	-2,147,483,648 to 2,147,483,647	%d or %i	Standard integers
unsigned int	2 or 4	0 to 4,294,967,295	%u	Positive integers
long int	4 or 8	±2,147,483,648 (min 4 bytes)	%ld	Large integers
unsigned long	4 or 8	0 to 4,294,967,295+	%lu	Large positive integers
long long	8	±9,223,372,036,854,775,808	%lld	Very large integers
float	4	±3.4E-38 to ±3.4E+38 (6 digits precision)	%f or %.2f	Decimal numbers
double	8	±1.7E-308 to ±1.7E+308 (15	%lf	High precision

		digits)		decimals
long double	10-16	Extended precision	%Lf	Very high precision
void	0	No value	-	Function returns, generic pointers

Format Specifier Details

Specifier	Type	Example Usage	Common Errors
%d	int	printf("%d", 42);	Using with float → garbage
%c	char	printf("%c", 'A');	Using with int → ASCII interpretation
%u	unsigned	printf("%u", 100);	Using with negative → wrap around
%ld	long	printf("%ld", 1234567890L);	Using %d with long → truncation
%f	float/double	printf("%f", 3.14);	Default: 6 decimal places
%.2f	float (2 decimals)	printf("%.2f", 3.14159);	Outputs: 3.14
%e	Scientific notation	printf("%e", 1234.5);	Outputs: 1.234500e+03
%x	Hexadecimal	printf("%x", 255);	Outputs: ff
%o	Octal	printf("%o", 8);	Outputs: 10
%p	Pointer address	printf("%p", &var);	Outputs: 0x7fff5694dc58

4. C Keywords (32 Total)

Keywords by Category

Category	Keywords	Purpose
Data Types	char, int, float, double, void, short, long, signed, unsigned	Declare variable types
Control Flow	if, else, switch, case, default, for, while, do, break, continue, goto, return	Program flow control
Storage Classes	auto, static, extern, register	Variable storage and scope
Type Modifiers	const, volatile	Modify type behavior
User-Defined	struct, union, enum, typedef	Create custom types

Special	sizeof	Get size of type/variable
---------	--------	---------------------------

Important Keyword Facts:

- All keywords are lowercase
- Cannot be used as identifiers
- Reserved by the compiler

5. Preprocessor Directives & Macros

Complete Preprocessor Directive Reference

Directive	Syntax	Purpose	Example
#include	#include <file.h> #include "file.h"	Include header files	#include <stdio.h>
#define	#define NAME value	Define constants/macros	#define PI 3.14159
#undef	#undef NAME	Undefine macro	#undef PI
#ifdef	#ifdef MACRO	Conditional compilation	#ifdef DEBUG
#ifndef	#ifndef MACRO	If not defined	#ifndef HEADER_H
#if	#if condition	Conditional compilation	#if VERSION > 2
#else	#else	Alternative branch	Used with #if/#ifdef
#elif	#elif condition	Else if branch	#elif VERSION == 2
#endif	#endif	End conditional	Closes #if/#ifdef
#error	#error "message"	Compilation error	#error "Not supported"
#pragma	#pragma directive	Compiler-specific	#pragma once

Macro Types

Object-like Macros:

```
#define MAX_SIZE 100
#define PI 3.14159
#define ERROR_MSG "An error occurred"
```

Function-like Macros:

```
#define SQUARE(x) ((x) * (x))
#define MAX(a,b) ((a) > (b) ? (a) : (b))
#define MIN(a,b) ((a) < (b) ? (a) : (b))
#define ABS(x) ((x) < 0 ? -(x) : (x))
```

Predefined Macros

Macro	Expands To	Example Output
-------	------------	----------------

__DATE__	Current date	"Oct 14 2025"
__TIME__	Current time	"16:30:00"
__FILE__	Current filename	"program.c"
__LINE__	Current line number	42
__STDC__	1 if ANSI C compliant	1

6. Operators - Complete Reference

MASTER OPERATOR PRECEDENCE TABLE

Priority	Operator	Description	Associativity
1	() [] -> .	Parentheses, Array subscript, Member access	Left to Right
2	++ -- (postfix)	Post-increment/decrement	Left to Right
3	++ -- (prefix) + - (unary) ! ~ * & (pointer) sizeof (type)	Pre-increment/decrement Unary plus/minus Logical NOT, Bitwise NOT Dereference, Address-of Size operator Type cast	Right to Left
4	* / %	Multiplication, Division, Modulus	Left to Right
5	+ -	Addition, Subtraction	Left to Right
6	<< >>	Left shift, Right shift	Left to Right
7	< <= > >=	Relational operators	Left to Right
8	== !=	Equality operators	Left to Right
9	&	Bitwise AND	Left to Right
10	^	Bitwise XOR	Left to Right
11		Bitwise OR	Left to Right
12	&&	Logical AND	Left to Right
13		Logical OR	Left to Right
14	?:	Ternary conditional	Right to Left
15	= += -= *= /= %= &= ^= = <<= >>=	Assignment Compound assignment	Right to Left

16	,	Comma operator	Left to Right
----	---	----------------	---------------

Arithmetic Operators

Operator	Operation	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2 (int)	2
/	Division	5.0 / 2 (float)	2.5
%	Modulus	5 % 2	1
++	Increment	x++ or ++x	x + 1
--	Decrement	x-- or --x	x - 1

Relational Operators

Operator	Meaning	Example	Result
<	Less than	3 < 5	1 (true)
<=	Less than or equal	3 <= 3	1 (true)
>	Greater than	3 > 5	0 (false)
>=	Greater than or equal	5 >= 5	1 (true)
==	Equal to	5 == 5	1 (true)
!=	Not equal to	3 != 5	1 (true)

Logical Operators Truth Tables

Logical AND (&&)

A	B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

Logical OR (||)

A	B	A B
0	0	0
0	1	1

1	0	1
1	1	1

Logical NOT (!)

A	!A
0	1
1	0

Bitwise Operators

Operator	Operation	Example	Binary Operation	Result
&	AND	5 & 3	0101 & 0011	0001 (1)
	OR	5 3	0101 0011	0111 (7)
^	XOR	5 ^ 3	0101 ^ 0011	0110 (6)
~	NOT (1's complement)	~5	~0101	1010 (-6)
<<	Left shift	5 << 1	0101 << 1	1010 (10)
>>	Right shift	5 >> 1	0101 >> 1	0010 (2)

Important Bitwise Facts:

- $\square$ Left shift by 1 = **Multiply by 2**
- $\square$ Right shift by 1 = **Divide by 2**
- $\square$ XOR of same numbers = **0**
- $\square$ XOR with 0 = **Same number**

Assignment Operators

Operator	Equivalent	Example	Result
=	Simple assignment	a = 5	a becomes 5
+=	a = a + b	a += 3	a = a + 3
-=	a = a - b	a -= 3	a = a - 3
*=	a = a * b	a *= 3	a = a * 3
/=	a = a / b	a /= 3	a = a / 3
%=	a = a % b	a %= 3	a = a % 3
&=	a = a & b	a &= 3	a = a & 3
=	a = a b	a = 3	a = a 3
^=	a = a ^ b	a ^= 3	a = a ^ 3
<<=	a = a << b	a <<= 2	a = a << 2
>>=	a = a >> b	a >>= 2	a = a >> 2

Conditional (Ternary) Operator

Syntax: `condition ? expression_if_true : expression_if_false`

Examples:

```
int max = (a > b) ? a : b;           // Find maximum
int abs = (x < 0) ? -x : x;         // Absolute value
char grade = (marks >= 60) ? 'P' : 'F'; // Pass/Fail
int largest = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c); // Largest of 3
```

7. Type Conversion & Type Casting

Implicit Type Conversion (Automatic)

Conversion Hierarchy (Low to High):

```
char → short → int → unsigned → long → unsigned long → float → double → long double
```

Rules:

- 1. Lower type → Higher type (automatic, no data loss)
- 2. Integer + Float = Float
- 3. Character treated as integer (ASCII value)

Examples:

```
int a = 5;
float b = 4.2;
float result = a + b; // 'a' automatically converted to 5.0
// result = 9.2
```

Explicit Type Casting (Manual)

Syntax: `(target_type) expression`

Common Casting Scenarios:

Scenario	Code	Result	Note
Float to Int	<code>int x = (int)3.7</code>	<code>x = 3</code>	Truncation (not rounding)
Int Division	<code>float r = (float)5/2</code>	<code>r = 2.5</code>	Cast before division
Character to Int	<code>int ascii = (int)'A'</code>	<code>ascii = 65</code>	ASCII value
Pointer Casting	<code>char *p = (char*)malloc(10)</code>	-	malloc returns void*

Type Conversion Warnings:

- **Data Loss:** float to int loses decimal part
- **Overflow:** Large value to smaller type
- **Sign Loss:** Signed to unsigned changes negative values

- **Precision Loss:** double to float reduces precision

8. Expression Evaluation Rules

MASTER EVALUATION ORDER

1. **Parentheses** `()`
2. **Unary operators** `++, --, +, -, !, ~`
3. **Arithmetic operators** `*, /, %` then `+, -`
4. **Shift operators** `<<, >>`
5. **Relational operators** `<, <=, >, >=`
6. **Equality operators** `==, !=`
7. **Bitwise operators** `&` then `^` then `|`
8. **Logical operators** `&&` then `||`
9. **Conditional operator** `?:`
10. **Assignment operators** `=, +=, -=, etc.`
11. **Comma operator** `,`

Complex Expression Examples

Expression	Step-by-Step Evaluation	Result
<code>3 + 5 * 2</code>	<code>3 + (5 * 2) → 3 + 10</code>	13
<code>(3 + 5) * 2</code>	<code>8 * 2</code>	16
<code>10 / 2 * 3</code>	<code>(10 / 2) * 3 → 5 * 3</code>	15
<code>5 > 3 && 2 < 4</code>	<code>(5 > 3) && (2 < 4) → 1 && 1</code>	1
<code>a = b = c = 5</code>	Right to left: <code>c=5, b=5, a=5</code>	<code>a=b=c=5</code>
<code>++a * b--</code>	Pre-increment a, multiply, post-decrement b	Depends on values

9. Common Errors & Solutions

Format Specifier Errors

Error	Wrong Code	Correct Code	Issue
Type Mismatch	<code>printf("%d", 3.14)</code>	<code>printf("%f", 3.14)</code>	Wrong specifier for float
Missing &	<code>scanf("%d", num)</code>	<code>scanf("%d", &num)</code>	Need address for scanf
Long vs Int	<code>printf("%d", longVar)</code>	<code>printf("%ld", longVar)</code>	Truncation possible
Unsigned	<code>printf("%d", -1U)</code>	<code>printf("%u", -1U)</code>	Sign interpretation

Operator Errors

Error	Wrong	Right	Explanation
Assignment vs Equality	<code>if (a = 5)</code>	<code>if (a == 5)</code>	Assignment always true

Integer Division	float x = 5/2	float x = 5.0/2	Result would be 2, not 2.5
Precedence	if (!x > 5)	if (!(x > 5))	! has higher precedence
Bitwise vs Logical	if (a & b)	if (a && b)	Different operations

10. Quick Reference Cards

Data Type Selection Guide

Use Case	Recommended Type	Reason
Character/ASCII	char	1 byte, efficient
Boolean (0/1)	int OR char	C doesn't have bool
Counter (small)	int	Standard, fast
Array index	int OR size_t	Non-negative
Money calculations	double	Precision needed
Scientific calc	double OR long double	High precision
Memory addresses	Pointers	Platform-specific
File sizes	long OR unsigned long	Can be large

Operator Shortcuts

Operation	Long Form	Short Form
Increment by 1	x = x + 1	x++ OR ++x
Add n	x = x + n	x += n
Toggle bit	x = x ^ 1	x ^= 1
Multiply by 2^n	x = x * 8	x << 3
Divide by 2^n	x = x / 4	x >> 2
Check if even	if (x % 2 == 0)	if (!(x & 1))
Check if odd	if (x % 2 == 1)	if (x & 1)
Swap values	temp=a; a=b; b=temp;	a^=b; b^=a; a^=b;

11. MCQ Practice Topics & Key Points

Module 1 - Critical MCQ Topics

Memory Segments:

- **Text segment** is READ-ONLY and SHARABLE
- **Initialized data** has TWO parts: read-only (const) and read-write
- **BSS** stores UNINITIALIZED globals/statics (initialized to 0)
- **Stack** grows DOWNWARD, Heap grows UPWARD

- `Local variables` have GARBAGE values by default
- `Static/Global uninitialized` have 0 by default

Compilation Process:

- **Order:** Source (.c) → Preprocessed (.i) → Assembly (.s) → Object (.o) → Executable (.exe)
- **Preprocessor** handles `#include`, `#define`, removes comments
- **Compiler** checks syntax, generates assembly
- **Linker** combines object files with libraries
- **Loader** loads program into RAM

Program Structure:

- `main()` is MANDATORY
- **Documentation section** is OPTIONAL
- **Link section** uses angle brackets `<>` for system headers, quotes `"` for user headers
- **Global variables** are accessible throughout the program

Module 2 - Critical MCQ Topics

Data Types Size & Range:

- **char:** 1 byte, -128 to 127 (signed), 0 to 255 (unsigned)
- **int:** 2 or 4 bytes (system dependent)
- **float:** 4 bytes, 6 digits precision
- **double:** 8 bytes, 15 digits precision
- **void:** 0 bytes, no value

Format Specifiers:

- `%d` for int, `%u` for unsigned
- `%ld` for long, `%lld` for long long
- `%f` for float/double, `%lf` for double in scanf
- `%.2f` for 2 decimal places
- `%c` for char, `%s` for string

Operator Precedence (Top to Bottom):

1. **Parentheses** `()` - HIGHEST
2. **Unary** `++, --, !, ~, +, -`
3. **Arithmetic** `*, /, %` then `+, -`
4. **Shift** `<<, >>`
5. **Relational** `<, <=, >, >=`
6. **Equality** `==, !=`
7. **Bitwise** `&` then `^` then `|`
8. **Logical** `&&` then `||`
9. **Ternary** `?:`
10. **Assignment** `=, +=, -=, etc.`
11. **Comma** `,` - LOWEST

Keywords:

- Total 32 keywords in C
- All keywords are LOWERCASE
- Cannot be used as identifiers
- Examples: `auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short,`

signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while

Variable Rules:

- `[]` Must start with LETTER or UNDERSCORE
- `[]` Cannot start with DIGIT
- `[]` Case SENSITIVE
- `[]` No special characters except UNDERSCORE
- `[]` Cannot be a KEYWORD

Type Conversion:

- `[]` **Implicit:** Automatic, lower to higher type
- `[]` **Explicit:** Manual using (type) casting
- `[]` **Hierarchy:** char < short < int < unsigned < long < float < double
- `[]` Integer division truncates: `5/2 = 2` (not 2.5)

Bitwise Operations:

- `[]` Left shift `<<` multiplies by 2^n
- `[]` Right shift `>>` divides by 2^n
- `[]` XOR same numbers = 0
- `[]` AND with 1 checks last bit (odd/even)

Logical Operators:

- `[]` `&&` returns 1 only if BOTH are true
- `[]` `||` returns 1 if AT LEAST ONE is true
- `[]` `!` inverts: `!0 = 1`, `!1 = 0`
- `[]` Short-circuit evaluation: `&&` stops at first false, `||` stops at first true

Preprocessor:

- `[]` Directives start with `#`
- `[]` `#define` creates compile-time constants (no memory)
- `[]` `const` creates runtime constants (uses memory)
- `[]` Predefined macros: `__DATE__`, `__TIME__`, `__FILE__`, `__LINE__`, `__STDC__`

12. Tricky MCQ Concepts

Common Tricks to Watch For:

Concept	Trick Question	Correct Answer	Explanation
Assignment in condition	<code>if (x = 5)</code>	Always true	Assignment returns value
Comma operator	<code>x = (5, 10)</code>	<code>x = 10</code>	Comma returns last value
Post vs Pre increment	<code>y = x++</code> VS <code>y = ++x</code>	<code>y=x</code> then <code>x++</code> VS <code>x++</code> then <code>y=x</code>	Order matters
Integer division	<code>float f = 5/2</code>	<code>f = 2.0</code>	Integer division first
Sizeof operator	<code>sizeof('a')</code>	4 (not 1)	Character constant is int

Array name	sizeof(array) VS sizeof(&array[0])	Total size vs pointer size	Array name context
Macro expansion	#define SQUARE(x) x*x SQUARE(2+3)	2+3*2+3 = 11	No parentheses
Unsigned overflow	unsigned int x = -1	4294967295	Wraps around

Associativity Tricks:

Expression	Evaluation	Result	Rule
a = b = c = 5	Right to left	All = 5	Assignment is R→L
*p++	*(p++)	Value then increment	Postfix higher than *
++*p	++(*p)	Increment value at p	Unary R→L
`a		b && c`	`a

13. Quick Test Checklist

Before the Test - Review These:

Module 1 Checklist:

- ☐ Can you draw the memory layout diagram?
- ☐ Know all 5 memory segments and what goes where?
- ☐ Remember compilation phases in order?
- ☐ Know what each compilation tool does?
- ☐ Remember program structure sections in order?
- ☐ Difference between library and user-defined functions?

Module 2 Checklist:

- ☐ All 32 keywords memorized?
- ☐ Data type sizes and ranges?
- ☐ Format specifiers for each data type?
- ☐ Operator precedence table (at least top 10)?
- ☐ Difference between / and % operators?
- ☐ Bitwise vs Logical operators?
- ☐ Type conversion hierarchy?
- ☐ Macro vs const difference?
- ☐ Variable naming rules?
- ☐ Truth tables for logical operators?

Common MCQ Patterns:

- "Which is NOT valid?" - Look for keyword usage, wrong syntax
- "What is the output?" - Trace through operator precedence
- "Size of this type?" - Remember system dependencies

4. "Which segment stores?" - Know memory layout
 5. "Correct format specifier?" - Match type to specifier
 6. "Result of expression?" - Apply precedence rules
 7. "Which statement is true?" - Check each option carefully
-

14. Last-Minute Formula Sheet

Quick Conversions:

- **Binary to Decimal:** Multiply each bit by 2^{position}
- **Left shift by n:** Multiply by 2^n
- **Right shift by n:** Divide by 2^n
- **Check even:** `(n & 1) == 0`
- **Check odd:** `(n & 1) == 1`
- **Toggle bit:** XOR with 1
- **Clear bit:** AND with 0
- **Set bit:** OR with 1

Memory Quick Facts:

- **1 byte** = 8 bits
- **1 KB** = 1024 bytes
- **char** = 1 byte always
- **int** = 2 or 4 bytes (system dependent)
- **pointer** = 4 bytes (32-bit) or 8 bytes (64-bit)

ASCII Values:

- **'0' to '9':** 48 to 57
 - **'A' to 'Z':** 65 to 90
 - **'a' to 'z':** 97 to 122
 - **Space:** 32
 - **Null '\0':** 0
-

15. Example Programs for Concepts

Memory Segments Example:

```
#include <stdio.h>
#include <stdlib.h>

int global_init = 100;           // Initialized data segment
int global_uninit;               // BSS segment (= 0)
const char msg[] = "Hello";     // Text segment (read-only)

int main() {                     // main() in text segment
    int local = 50;              // Stack
    static int st = 60;          // Initialized data segment
    int *ptr = malloc(4);        // Heap allocation

    printf("Address of global_init: %p (Data)\n", &global_init);
    printf("Address of global_uninit: %p (BSS)\n", &global_uninit);
    printf("Address of local: %p (Stack)\n", &local);
```

```

printf("Address of malloc: %p (Heap)\n", ptr);

free(ptr);
return 0;
}

```

Operator Precedence Example:

```

#include <stdio.h>

int main() {
    int a = 5, b = 3, c = 2;

    // Arithmetic precedence
    printf("5 + 3 * 2 = %d\n", 5 + 3 * 2);          // 11
    printf("(5 + 3) * 2 = %d\n", (5 + 3) * 2);      // 16

    // Relational and logical
    printf("5 > 3 && 2 < 4 = %d\n", 5 > 3 && 2 < 4); // 1

    // Ternary operator
    int max = (a > b) ? a : b;
    printf("Max of %d and %d = %d\n", a, b, max);

    // Bitwise operations
    printf("5 << 1 = %d (multiply by 2)\n", 5 << 1); // 10
    printf("5 >> 1 = %d (divide by 2)\n", 5 >> 1);   // 2
    printf("5 & 1 = %d (check odd)\n", 5 & 1);       // 1 (odd)

    return 0;
}

```

FINAL TEST TIPS

1. **Read questions carefully** - Look for NOT, EXCEPT, INCORRECT
2. **Check data types first** - Many errors come from type mismatches
3. **Trace operator precedence** - Use parentheses mentally
4. **Remember defaults** - Local = garbage, Global/Static = 0
5. **Format specifiers** - Match exactly with data type
6. **Keywords** - All lowercase, cannot be identifiers
7. **Memory segments** - Stack (local), Heap (dynamic), Data/BSS (global/static)
8. **Compilation order** - .c → .i → .s → .o → .exe
9. **Type conversion** - Always upward in hierarchy unless explicitly cast
10. **Bitwise tricks** - Left shift = multiply, Right shift = divide

Best of luck with your test! You've got this! 🍀

Remember: These notes cover ALL topics from both modules. Focus on understanding concepts rather than memorization where possible, as MCQs often test application of knowledge.